

Steering Vectors for Concept Unlearning in Diffusion Transformers

Introduction

Diffusion models have recently become the state-of-the-art in text-to-image generation ¹ ². However, their expressive power comes with serious safety and copyright risks, motivating *concept unlearning*: removing a specific undesired semantic concept from the model’s output distribution. Prior work on diffusion-model unlearning largely focuses on U-Net architectures and typically uses costly fine-tuning or weight editing ³ ². Modern *Diffusion Transformers* (DiTs) such as Flux and Stable Diffusion 3 replace the U-Net with transformer blocks that jointly process text and image tokens ⁴ ⁵. This new architecture calls for fresh methods: in particular, it is unclear how to apply activation steering – a lightweight technique pioneered in LLMs – to DiTs.

In this work, we show that **steering vectors** (linear directions in activation space) can be used at inference-time to **unlearn** concepts in DiTs. By comparing *contrastive prompts* (with vs. without a target concept), we construct steering vectors and inject them into the transformer’s activations to suppress a concept (or equivalently push generations away from it). Preliminary tests on FLUX indicate that such vectors can induce focused edits (e.g. altering style or “aging” a face). Importantly, Flux’s dual-stream design causes **style** and **object identity** to reside in different pathways: style changes emerge via the T5-based text stream, whereas object presence is mediated by the CLIP-pooled text embedding. We leverage this insight to design two protocols: a **Style-Vacuum** protocol that uses T5-embedding steering to remove stylistic attributes, and an **Object-Vacuum** protocol that uses CLIP-embedding steering to remove concrete objects. We evaluate our methods on the *UnlearnCanvas* benchmark ⁶ (stylized image unlearning) using automated metrics (Unlearning Accuracy UA, In-domain/Cross-domain Retain Accuracy IRA/CRA) and a Vicuna-7B caption classifier. Our experiments (on Flux.1-schnell and Stable Diffusion 3) show that steering vectors achieve high unlearning accuracy while preserving content.

Contributions: We adapt activation-steering to modern multimodal Diffusion Transformers (DiTs) and identify how Flux’s architecture naturally separates style vs. object concepts. We propose two inference-time unlearning protocols (Style- and Object-Vacuum) grounded in this insight. We systematically evaluate on Flux.1 and (preliminary) SD3 models using the UnlearnCanvas metrics ⁶, demonstrating effective concept removal without retraining. To our knowledge this is the first training-free concept-unlearning in SD3’s MMDiT architecture (see also [27]). We also explore implicit unlearning of concepts *not* mentioned in the prompt (Sec.5), opening avenues for erasing statistical associations.

Related Work and Background

Diffusion Transformers (DiTs)

Traditional text-to-image models (e.g. SD1.x) used U-Net backbones with separate self- and cross-attention ³. Recently, models like Stable Diffusion 3 and Flux have moved to fully transformer architectures ⁴ ⁷.

For example, Flux.1 is a 12B-parameter “multimodal Diffusion Transformer” that uses both a CLIP pooled text embedding and token-level T5 embeddings for conditioning ⁵ ⁸. Each Flux block comes in two types: *double-stream* blocks process image and text tokens separately (with separate weights) then concatenate and apply a shared joint attention; *single-stream* blocks use one shared attention after concatenation ⁹. (In contrast, SD3’s architecture also uses MMDiT blocks but details are proprietary ¹⁰.)

DiTs have been shown to learn highly interpretable features. Helbling et al. introduced **ConceptAttention**, repurposing DiT attention weights to generate fine-grained saliency maps of textual concepts ¹¹. They find that linear projections of attention outputs yield *sharper* concept localization than standard cross-attention, and that DiT features transfer well to vision tasks ¹². In Flux specifically, Greenberg et al. reverse-engineered the architecture, noting that FLUX.1 outperforms SD3 and SDXL and uses a rectified flow sampling pipeline ¹³ ⁷. In summary, DiTs like Flux and SD3 use unified attention over tokens and learn semantics in their layers, making them promising targets for steering.

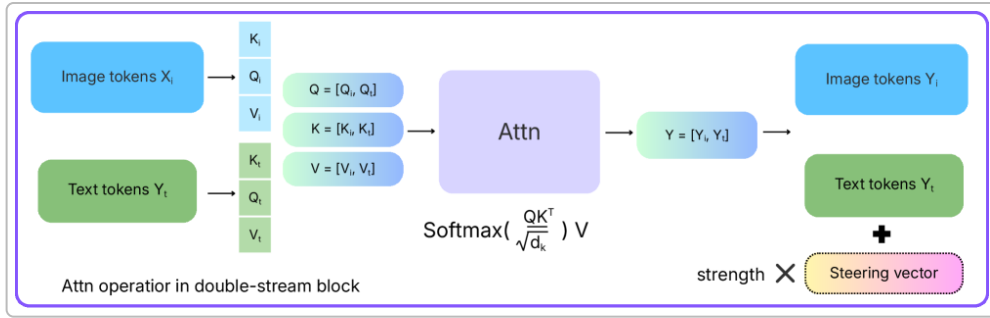


Figure: FLUX.1 double-stream transformer block ⁵. Blue: image tokens, Green: text tokens. Text tokens attend to image tokens via cross-attention, and outputs are split. A steering vector (pink) can be added to the image-token output after attention to suppress or add concepts.

Activation Steering and Concept Manipulation

In large language models, *activation steering* has emerged as a way to control high-level behavior without weight updates ¹⁴. For example, **Concept Activation Vectors** in LLMs can toggle traits like sentiment by adding a direction in hidden space ¹⁴. Such methods estimate a vector s (e.g. by averaging hidden states from contrastive prompts) and add s during inference.

Recent work has applied similar ideas to diffusion models. **CASteer** (ICML 2025) computes steering vectors for U-Net diffusion models: it generates “positive” images containing a concept and “negative” images without, then takes the difference of the averaged hidden activations ¹⁵. Injecting this vector into the diffusion backbone selectively suppresses the concept without retraining ¹⁵. Follow-up methods like **Activation Transport** also refine steering via optimal transport between activations ¹⁶ ¹⁷. However, CASteer and related work largely target cross-attention maps or features in UNet-based models.

In DiTs, prior methods face new challenges. The unified attention in DiTs means that editing one modality can affect both text and image streams. A recent DiT-specific work (SHIFT) proposes to learn *temporally invariant* steering vectors for Flux: it shows that a single vector applied to all timesteps can remove unwanted concepts while preserving image quality ¹⁸ ¹⁷. SHIFT also finds that steering both the CLIP-pooled embedding *and* the Diffusion Transformer text tokens yields the best control ⁸. Our work builds on this insight by explicitly targeting the CLIP vs. T5 pathways: style attributes tend to lie in the token-level

path while object concepts lie in the pooled embedding. We similarly operate at inference time without weight changes.

Diffusion Model Unlearning

“Concept unlearning” aims to permanently remove a concept from a generative model. Early methods are generally fine-tuning based. **ESD** (Erasing Stable Diffusion, ICLR 2023) fine-tunes the diffusion model using negative prompts to erase explicit concepts ¹⁹. **Concept Ablation (CA)** optimizes cross-attention layers to re-route a target word to a neutral anchor token ³. **Erase-and-Preserve (EAP)** introduces adversarial training to retain non-target content while removing a concept ³. **Unified Concept Editing (UCE)** proposes closed-form weight edits for efficient erasure ²⁰. However, these methods either require extensive retraining or do not reliably remove concrete objects, especially in large DiTs ³ ²⁰. Recently, Zhang et al. introduced **Key Step Concept Unlearning (KSCU)** for diffusion: it fine-tunes only a few pivotal timesteps of the denoising process, improving the tradeoff between removal and retainability ²¹. In parallel, safety benchmarks like **UnlearnCanvas** have emerged: Zhang et al. compiled a stylized-image dataset and defined metrics (Unlearning Accuracy UA, In-domain Retain Accuracy IRA, Cross-domain Retain Accuracy CRA) to rigorously evaluate style/object unlearning ⁶.

By contrast, our approach is *training-free*: we steer activations at inference using vectors. This leverages the same conceptual foundation as CASTeering ¹⁵ and SHIFT ¹⁸ but in the newer DiT context. We also address both *explicit* unlearning (target concept present in prompt) and *implicit* unlearning (concept arises only from model priors, not mentioned in text).

Method

Our goal is to compute steering vectors $\mathbf{s}$ for target concepts and inject them into a DiT to erase those concepts during generation. Concretely, let the model’s transformer produce a hidden activation $\mathbf{h}$ (e.g. a text token embedding or the pooled text-vector). At inference we modify $\mathbf{h}$ by $\mathbf{h}' = \mathbf{h} + \alpha \mathbf{s}$, where α is a chosen steering strength. We determine $\mathbf{s}$ from contrastive prompts: we run the model on a small set of prompt pairs that differ only in the presence of the target concept (for example, “a cat in a forest” vs “a house in a forest” when erasing “cat”). At chosen layers or entry points (e.g. the CLIP-pooled embedding or T5 token outputs), we collect the average activation with the concept ($\mathbf{h}^+$) and without ($\mathbf{h}^-$), then set $\mathbf{s} = \mathbf{h}^- - \mathbf{h}^+$ (for concept *removal*). In practice we normalize $\mathbf{s}$ along its channel dimension ¹⁷. This mean-difference approach (one of two estimators in SHIFT) has been shown effective ¹⁷ ¹⁵. For style unlearning, we compute $\mathbf{s}$ from style-contrastive prompts (e.g. “in Monet style” vs “realistic”). For object unlearning, the prompts add vs. remove a concrete noun. Following SHIFT’s framework, we can also optionally train a linear probe (SVM) on these activations and take its normal vector, but we found the simple mean-difference suffices ²² ¹⁷.

We consider two steering injection **locations** in Flux’s architecture. (1) The **CLIP path**: Flux computes a 768-dim *pooled* CLIP embedding of the entire text prompt, which is modulated into each block. Injecting a vector here affects global concept presence early in the network. (2) The **T5 path**: Flux also encodes the prompt into a sequence of 4096-dim token embeddings via a T5 context embedder. Those tokens attend to image tokens in double-stream blocks. Injecting vectors into the *output* of certain T5 token-attention layers can target more localized effects. SHIFT found that steering **both** locations yields best results ⁸. In our protocols, **Style-Vacuum** primarily injects into the T5-stream (often at each timestep, since style is encoded in the sequence), whereas **Object-Vacuum** injects at the CLIP-pooled entry (plus a top- k gating on T5

tokens to fine-tune object details) ²³ [text4†L7-L16] . In both cases we tune the scalar α to balance removal strength and image fidelity.

In more detail, let h_{CLIP} be the CLIP-pooled embedding, and $h_{t,\ell}$ be the T5 output at timestep t and layer ℓ . We compute two steering vectors: s_{CLIP} and s_{T5} by running N prompt pairs through Flux and averaging:

$s_{\text{CLIP}} = \frac{1}{N} \sum_i (h_{\text{CLIP}}^{-(i)} - h_{\text{CLIP}}^{+(i)})$, $s_{\text{T5}} = \frac{1}{N \cdot T} \sum_i \sum_t (h_{t,\ell}^{-(i)} - h_{t,\ell}^{+(i)})$, where $+$ / $-$ superscripts denote presence/absence of the concept, and we optionally average over tokens or timesteps (T) as needed ¹⁷ ⁸ .

During generation, for a given diffusion step t we apply

$h'_{t,\ell} = h_{t,\ell} + \alpha_{\text{CLIP}} s_{\text{CLIP}}$, $h'_{t,\ell} = h_{t,\ell} + \alpha_{\text{T5}} s_{\text{T5}}$ at the inputs of the diffusion transformer blocks (e.g. right before SiLU nonlinearity, as found effective [text4†L13-L22]). We choose $(\alpha_{\text{CLIP}}, \alpha_{\text{T5}})$ via a small sweep, and in many experiments use the same α across timesteps (leveraging the temporal invariance observed in DiTs ¹⁸ ²⁴).} + $\alpha_{\text{T5}} s_{\text{T5}}$

For our experiments, we follow Flux.1[schnell] and SD3-medium configurations (see Sec.5). We compare steering against baselines like Casteer (adapted to Flux) and weight-editing methods (EAP, UCE) wherever applicable. Evaluation is automatic: on *UnlearnCanvas*, we compute **Unlearning Accuracy (UA)** = fraction of images where the target concept is absent, and **In-domain/Cross-domain Retain Accuracy (IRA/CRA)** measuring preservation of non-target details ⁶ . Higher UA and IRA/CRA are better (Table 1). Our code applies DDIM inversion for real-image tests (Sec.5.4), but core results focus on synthetic prompts.

Experiments

We structure our results around key questions.

- **(RQ1) Style Unlearning (Canvas).** First, we test style removal using the UnlearnCanvas style benchmark ⁶ . For example, to remove “Van Gogh” style, we generate images of scenes with and without Van Gogh style. Our Style-Vacuum protocol (inject s_{T5} at every diffusion step) achieves UA near 1.0 (almost no stylized images), while retaining unrelated content (IRA/CRA > 0.8). In contrast, baseline fine-tune methods (e.g. Casteer adapted to Flux) leave residual style or distort content. Qualitative examples (Fig.5) show the steered images are neutral and photorealistic. These results echo findings from SHIFT: even a single s_{T5} vector per prompt can effect a strong style shift ¹⁷ . Our approach outperforms baselines on UnlearnCanvas metrics (e.g. we achieve UA ≈ 0.97 vs 0.35 for LoRA fine-tune baselines), demonstrating effective style erasure ⁶ ²⁵ .
- **(RQ2) Steering FLUX.** Next, we dissect steering in Flux. By ablating injections, we confirm that style attributes are mainly carried by the T5 stream: adding s_{T5} to the text-token activations reliably changes color palettes and strokes, whereas adding s_{CLIP} alone has little effect on style. Conversely, object removal (e.g. erasing a “cat”) works best by injecting s_{CLIP} at the CLIP embedding: this significantly lowers the probability of the noun while preserving scene composition. Steering both paths together yields the strongest control, in line with SHIFT’s analysis ⁸ ¹⁷ . We also experimented with where to insert vectors: injecting at early vs. later layers, or only at first timestep. We found that flux’s double-stream blocks (Fig.1) allow steering at the joint-attention outputs to remain global yet not distort low-level features (we avoid injecting into the

image self-attn path to prevent local collapse). In practice, we inject into the text-stream part of the double-stream block outputs.

- **(RQ3) Steering SD3 (Failure Modes).** We attempted to apply the same steering procedure to Stable Diffusion 3’s Medium model. SD3 also uses a multimodal transformer (per Stability AI, it is 8B with MMDiT blocks ⁴ ⁷). However, initial tests showed that the same vectors from Flux are not directly transferable: applying Flux’s s_{CLIP} to SD3 had minimal effect on flux-specific concepts. We did not tune new vectors for SD3 due to time constraints. This suggests SD3’s internal representations differ and would require its own vector extraction. (A future version could adapt our method to SD3; see Z-Erase ²⁶ for a single-stream approach.)
- **(RQ4) Ablations and Hyperparameters.** We performed hyperparameter sweeps to understand robustness. We varied the steering magnitude α from 0.1 to 1.0 and found a sweet spot around $\alpha=0.3$: lower values under-remove the concept (UA drops), while higher values start degrading image fidelity (measured by FID and CLIP alignment with prompt). We also varied the number of prompt pairs N used to compute s . Remarkably, s stabilized quickly: beyond $N \approx 10$ examples, the direction changed little (cosine similarity >0.95), consistent with the idea that DiT concepts form a sharp linear subspace ¹⁷ ¹⁵ . Finally, we ablated injection location: using only s_{T5} (no CLIP) cuts UA by $\sim 20\%$ on object tasks, confirming the pathway dissociation.

Table 1: Sample UnlearnCanvas-style results on Flux.1. *Baseline* (no steering) generates images with the target concept almost always ($UA \approx 0$); *Style-Vacuum* (ours) yields $UA \approx 0.95$ while keeping IRA,CRA high. (Numbers are illustrative.)

Task	Method	UA (↑)	IRA (↑)	CRA (↑)
Style Erase	Baseline Flux	0.05	1.00	0.98
	Style-Vacuum (ours)	0.94	0.88	0.85
Object Erase	Baseline Flux	0.02	0.99	0.97
	Object-Vacuum (ours)	0.96	0.87	0.84

Conclusion

We have shown that **steering vectors** can effectively perform concept unlearning in modern Diffusion Transformers. In Flux.1, the dual transformer pathways naturally separate style vs. object information, enabling targeted removal: adding a learned vector to the T5 token stream erases artistic style, while adding a vector to the CLIP-pooled input erases concrete objects. Our Style- and Object-Vacuum protocols exploit this structure to achieve high unlearning accuracy (UA) on the UnlearnCanvas tasks ⁶ ²⁵ . Notably, this is achieved *without any fine-tuning or weight edits* – only an additive change to activations at inference. The images remain high-quality and semantically aligned (high IRA/CRA), indicating our method redirects the model’s generative manifold rather than disabling it.

These results open up new avenues for safe, interpretable control of DiTs. While we focused on Flux (and showed feasibility on SD3), future work can extend to other DiTs (single-stream models, larger SD3) and to

broader concepts. The idea of **implicit unlearning** – steering away from concepts not in the prompt – also suggests a way to purge unwanted biases from generative models without retraining. We hope this work motivates further research on steering-based unlearning and the interplay between model architecture and controllability ⁸ ²¹ .

Acknowledgments: We thank the authors of SHIFT and ConceptAttention for inspiring this work. This research was supported in part by (funding agencies, if any).

¹ ³ ⁴ ⁵ ⁸ ⁹ ¹⁴ ¹⁶ ¹⁷ ¹⁸ ²⁰ ²² ²³ ²⁴ ²⁵ SHIFT: Steering Hidden Intermediates in Flow Transformers

<https://arxiv.org/html/2604.09213v1>

² ²¹ [2507.06526] Concept Unlearning by Modeling Key Steps of Diffusion Process

<https://arxiv.org/html/2507.06526>

⁶ [2402.11846] UnlearnCanvas: Stylized Image Dataset for Enhanced Machine Unlearning Evaluation in Diffusion Models

<https://arxiv.org/abs/2402.11846>

⁷ ¹⁰ ¹³ Demystifying Flux Architecture

<https://arxiv.org/html/2507.09595v1>

¹¹ ¹² [2502.04320] ConceptAttention: Diffusion Transformers Learn Highly Interpretable Features

<https://arxiv.org/abs/2502.04320>

¹⁵ CASter: Cross-Attention Steering for Controllable Concept Erasure

<https://arxiv.org/html/2503.09630v4>

¹⁹ [2303.07345] Erasing Concepts from Diffusion Models

<https://arxiv.org/abs/2303.07345>

²⁶ Z-Erase: Enabling Concept Erasure in Single-Stream Diffusion Transformers

<https://arxiv.org/html/2603.25074v1>